SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: **Theory of Computation** **COURSE CODE: CSA13**

COURSE OUTCOME COVERED

CO1: *Apply mathematical foundations such as formal proofs, induction, and basic concepts of automata theory to solve computational problems. (BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Duration : 1 Hour

Simulation

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed*, *Opening*, *Open*, and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

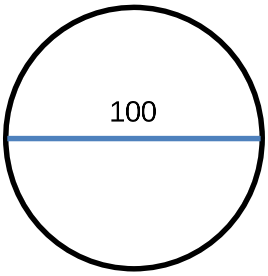
3 Design DFA using simulator to accept the string the end with ab over set {a,b}

W= aaabab

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

RUBRICS FOR CALCULATION



1. NFA for Automatic Door System

States

- $q0$ – Closed (start)
- $q1$ – Opening
- $q2$ – Open (Accepting)
- $q3$ – closing

Input symbols

$\Sigma = \{P, N\}$

- P = Person detected
- N = No person detected

Transition Table

Current State	P	N
$\rightarrow q0$ (closed)	$\{q1\}$	$q0$
$q1$ (opening)	$q2$	$q2$
$^*q2$ (Open)	$\{q2\}$	$\{q2, q3\}$
$q3$ (closing)	$q1$	$q0$

2. DFA for Traffic Light System

States

- $q0$ = Red (start)
- $q1$ = Green
- $q2$ = Yellow

Input

T = Timer

Transition Table

Current State	T
$\rightarrow q0$ (Red)	$q1$
$q1$ (Green)	$q2$
$q2$ (Yellow)	$q0$

Extensions

- Pedestrian Signal : Add state $q3$ (Pedestrian crossing)
- Emergency Vehicle : Add emergency state $q4$ where all lights remain Red except the emergency lane.

3. DFA to Accept Strings Ending with "ab"

Alphabet : $\Sigma = \{a, b\}$

Given string : W = aaabab

Since it ends with ab, it is Accepted.

States

- q0 – Start
- q1 – Last symbol was 'a'
- q2 – Ends with "ab" (Accept)

Transition Table

State	a	b
→q0	q1	q0
q1	q1	q2
*q2	q1	q0

Simulation for aaabab

Input	State
Start	q0
a	q1
a	q1
a	q1
b	q2
a	q1
b	q2 ✓

Result : Accepted

4. DFA Accepting Only "c", "bc", and "bcaaa"

States

q0 - start

q1 = b

q2 = bc (accept)

q3 = bca

q4 = bcaa

q5 = bcaaa (accept)

q6 = c (accept)

qd = Dead

Transition Table

State	a	b	c
→ q0	qd	q1	q6*
q1	qd	qd	q2*
*q2	q3	qd	qd
q3	q4	qd	qd

q4	q5*	qd	qd
*q5	qd	qd	qd
*q6	qd	qd	qd
qd	qd	qd	qd

5. DFA Accepting All Strings Beginning with a or b

Alphabet : $\Sigma = \{a, b\}$

Language : All non-empty strings over $\{a, b\}$

States

q^0 = start

q^1 = Accept

qd = Dead (optional for ε)

Transition Table

State	a	b
$\rightarrow q_0$	q_1	q_1
$*q_1$	q_1	q_1
q_d	q_d	q_d